Linguaggi di Programmazione

Bottom-up parser

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Parser Bottom-up (o shift-reduce)

Parser Bottom-up (o shift-reduce)

Due tipi di operazioni elementari:

- **Shift**: un simbolo terminale viene spostato dall'input sulla pila.
- **Reduce**: una serie di simboli (terminali e nonterminali) sulla cima della pila corrisponde al "reverse" di una parte destra di una produzione $A \to \alpha \in R$ (α^R sulla pila). La stringa α^R viene rimossa dalla pila e sostituita con A ("α^R viene ridotta ad A").

Riprendiamo la presentazione di un parser shift-reduce nondeterministico, che è essenzialmente un PDA con un solo stato che riconosce per pila vuota il linguaggio $L\$ = \{w\$ \mid w \in L\}$.

Questi parser vengono anche detti **Parser LR**:

- L (leggo da sinistra (left) a destra)
- R (derivazione rightmost)

Parser shift-reduce nondeterministico

Input: una grammatica libera G con simbolo iniziale S ed una stringa $w \in T^*$

Output: se $w \in L(G)$, allora restituisce la sua derivazione rightmost a rovescio, cioè con S alla fine sulla pila.

- Inizializza la pila a \$.
- Inizializza l'input a w\$.
- Usa il PDA $M = (T, \{q\}, T \cup NT \cup \{\$\}, \delta, \$, \emptyset)$ per riconoscere w\$ per pila vuota, dove la funzione di transizione δ è definita come segue:
  - $(q, aX) \in \delta(q, a, X) \qquad \forall a \in T, \forall X \in T \cup NT \cup \{\$\} \qquad$ (shift)
  - $(q, A) \in \delta(q, \epsilon, \alpha^R) \qquad$ se $A \to \alpha \in R \qquad$ (reduce)
  - $(q, \epsilon) \in \delta(q, \$, S\$) \qquad$ (accept)
- Ogni volta che facciamo una reduce, forniamo in output la produzione usata.
- Alla fine, S\$ sulla pila con \$ in input $\Rightarrow$ ok, accettiamo.

Esempio: grammatica delle espressioni aritmetiche

$E \to T \mid T + E \mid T - E$

$T \to A \mid A * T$

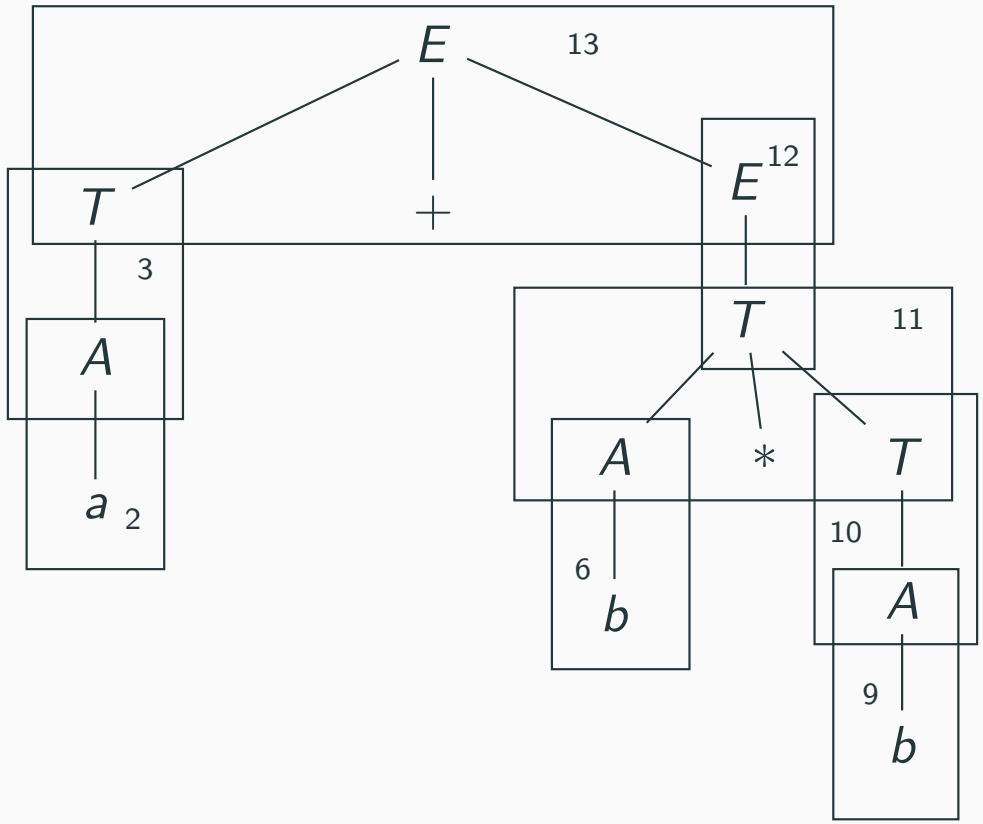
$A \to a \mid b \mid \epsilon$

Step	Stack	Input	Azione
1	$\$$	$a + b * b\$$	shift
2	$\$a$	$+b * b\$$	reduce $A \to a$
3	$\$A$	$+b * b\$$	reduce $T \to A$
4	$\$T$	$+b * b\$$	shift
5	$\$T+$	$b * b\$$	shift
6	$\$T + b$	$*b\$$	reduce $A \to b$
7	$\$T + A$	$*b\$$	shift
8	$\$T + A*$	$b\$$	shift
9	$\$T + A * b$	$\$$	reduce $A \to b$
10	$\$T + A * A$	$\$$	reduce $T \to A$
11	$\$T + A * T$	$\$$	reduce $T \to A * T$
12	$\$T + T$	$\$$	reduce $E \to T$
13	$\$T + E$	$\$$	reduce $E \to T + E$
14	$\$E$	$\$$	accept
15	ϵ	ϵ	(fine)

Esempio

La derivazione rightmost a rovescio si legge (dalla tabella sopra) mettendo insieme lo stack (che cresce verso destra) e l'input:
$a + b * b \Leftarrow A + b * b \Leftarrow T + b * b \Leftarrow T + A * b \Leftarrow T + A * A \Leftarrow T + A * T \Leftarrow T + T \Leftarrow T + E \Leftarrow E$

Costruzione dell'albero bottom-up!



Nondeterminismo

Il parser è nondeterministico perché

- chi ha precedenza tra shift e reduce?
  - 2) $\$a \quad + b * b\$$ potrei fare shift, e ottenere
  - $3')$ $\$a + \quad (b)\$$ scelta infruttuosa! Qui reduce è meglio di shift.
- Altro esempio su: chi ha precedenza tra shift e reduce?
  - 4) $\$T \quad + b * b\$$ potrei fare reduce $E \to T$, e ottenere
  - $5')$ $\$E \quad + b * b\$$ scelta infruttuosa! Qui shift è meglio di reduce.
- chi ha precedenza tra due diverse reduce?
  - 11) $\$T + A * T \quad \$$ se faccio reduce $T \to T * A$, ottengo
  - 12) $\$T + T \quad \$$ oppure, se faccio reduce $E \to T$, ottengo
  - $12')$ $\$T + A * E \quad \$$ Scelta infruttuosa! Qui reduce $T \to T * A$ è meglio di reduce $E \to T$.

Ci sono conflitti che vanno risolti, possibilmente in modo deterministico.

Come risolvere i conflitti?

È buona norma scegliere l'azione in modo tale che ciò che si trova sulla pila sia un prefisso (a rovescio) di una parte destra di una produzione della grammatica. Ad esempio, qui scelgo 3):

- 3′) $\$a+$ nessuna produzione ha reverse che comincia con $a+$
- 3) $\$A$ c'è una produzione il cui reverse comincia con A ($T \to A$)

Mentre qui scelgo 12):

- 12′) $\$T + A * E$ non è un prefisso di parte destra di una produzione,
- 12) $\$T + T$ invece lo è.

Strategia: bisogna scegliere l'azione giusta in modo che sulla pila ci sia un prefisso viabile. Intuitivamente,

Prefisso viabile: è una sequenza $\in (T \cup NT)^*$ che può apparire sulla pila di un parser bottom-up per una computazione che accetta un input.

Prefisso viabile

Data una grammatica libera $G = (NT, T, S, R)$ e una sua **derivazione rightmost**

$$S \Rightarrow_r^* \alpha A w \Rightarrow_r \alpha \beta w$$

con $w \in T^*, \alpha \in (T \cup NT)^*$ e $A \to \beta \in R$, diciamo che $\alpha\beta$ è un **prefisso viabile completo**. β è detta maniglia (handle) della riduzione data dalla produzione $A \to \beta$ (perché β^R si trova in cima alla pila). Ogni prefisso di un prefisso viabile completo è un **prefisso viabile**.

Avvertenza

Data una grammatica libera $G = (NT, T, S, R)$, noi utilizzeremo la grammatica aumentata $G' = (NT \cup \{S'\}, T, S', R \cup \{S' \to S\})$ per costruire il parser bottom-up. Per tale grammatica, S è un prefisso viabile completo perché $S' \Rightarrow_r S$. Questo perché quando potremo fare questa riduzione e il prossimo carattere in input sarà \$, allora la stringa verrà accettata.

Il linguaggio dei prefissi viabili è regolare!

Come fare a

- riconoscere le maniglie sulla cima della pila e ridurle?
- scegliere tra più riduzioni solo quelle che producono sulla pila un nuovo prefisso viabile?
- scegliere spostamenti che completino il prefisso viabile sulla pila e facciano comparire una nuova maniglia?

Per risolvere questi problemi, può risultare utile il seguente teorema.

Teorema
Data $G = (NT, T, S, R)$ libera, i prefissi viabili della grammatica aumentata $G' = (NT \cup \{S'\}, T, S', R \cup \{S' \to S\})$ costituiscono un linguaggio regolare e, quindi, può essere descritto con un DFA in cui

- tutti gli stati sono di accettazione (perché ogni prefisso di un prefisso viabile è un prefisso viabile)
- e in cui ci sono stati da cui non esce nessuna transizione (corrispondenti al raggiungimento di un prefisso viabile completo).

DFA dei prefissi viabili

N.B.: Il DFA dei prefissi viabili non è esattamente un DFA perché la sua funzione di transizione δ non è totale.

Il parser può consultare il DFA dei prefissi viabili per decidere cosa fare:

1. Se la pila contiene un prefisso viabile completo, allora il paser riduce.
2. Se la pila contiene un prefisso viabile incompleto, allora il parser shifta.
3. Se la pila non contiene un prefisso viabile, allora errore.

A seconda di come è fatto il DFA dei prefissi viabili, il parser può risultare deterministico o meno, e può eventualmente sfruttare informazioni di look-ahead e sui follow dei nonterminali per ottenere determinismo nella scelta dell'azione da eseguire.

Serve la pila degli stati del DFA dei prefissi viabili

Anche se, concettualmente, ad ogni modifica della pila, il DFA dei prefissi viabili la riscandisce da capo per determinare come sia fatto il prefisso viabile corrente, questo non è necessario, perché ogni prefisso di un prefisso viabile è un prefisso viabile a sua volta!

Infatti, la pila viene modificata in due modi:

- Shift del simbolo x: la pila passa da $\$\gamma$ a $\$\gamma x$. In tal caso, il DFA si ritrova nello stato s dopo avere elaborato $\$\gamma$, quindi basta far ripartire il DFA da s con input x per determinare il nuovo stato.
- Reduce $A \to \alpha$: la pila passa da $\$\gamma\alpha$ a $\$\gamma A$. Non c'è bisogno di fare ripartire il DFA dalla base alla pila; basta ripristinare lo stato in cui si trovava subito prima di elaborare il primo simbolo di α (cioè dopo aver elaborato $\$\gamma$) e fornirgli il simbolo A in input.

Quindi mi serve uno stack con gli stati del DFA dei prefissi viabili! (E mi basta questo stack di stati, perché implicitamente rappresenta anche la pila dei simboli, come vedremo)

Parser LR(0)

Item LR(0)

Uno stato del DFA dei prefissi viabili (chiamato **automa canonico LR(0)**) è costituito da un insieme di **item LR(0)**.
Un item LR(0) è una produzione con indicata, con un punto, una posizione della sua parte destra. Ad esempio, $A \to XYZ$ genera 4 item

1. $A \to \cdot XYZ$
2. $A \to X \cdot YZ$
3. $A \to XY \cdot Z$
4. $A \to XYZ \cdot$

Il punto $\cdot$ indica quanta parte della produzione è già stata analizzata:

- Se $A \to \alpha \cdot \beta$ è nello stato del DFA in cima alla pila degli stati, allora vuol dire che α è sulla pila dei simboli e ci si aspetta che l'input da leggere contenga (o possa venire ridotto a) β.
- Se $A \to \alpha \cdot$ è nello stato del DFA in cima alla pila degli stati, allora sulla pila dei simboli c'è la maniglia α^R e possiamo fare la reduce di α^R a A.

N.B.: la produzione $A \to \epsilon$ genera l'item $A \to \cdot$

Automa Canonico LR(0)

Il DFA dei prefissi viabili, detto automa canonico LR(0), viene costruito usando le funzioni ausiliarie $Clos(I)$ e $Goto(I, X)$ dove I è un insieme di item e $X \in T \cup NT$.

```
def Clos(I):
    ripeti fino a quando I viene modificato:
        per ogni item A → α · Xβ ∈ I:
            per ogni produzione X → γ:
                aggiungi X → · γ ad I
    return I

def Goto(I, X):
    inizializza J = ∅
    per ogni item A → α · Xβ ∈ I:
        aggiungi A → αX · β a J
    return Clos(J)
```

Costruzione dell'Automa Canonico LR(0)

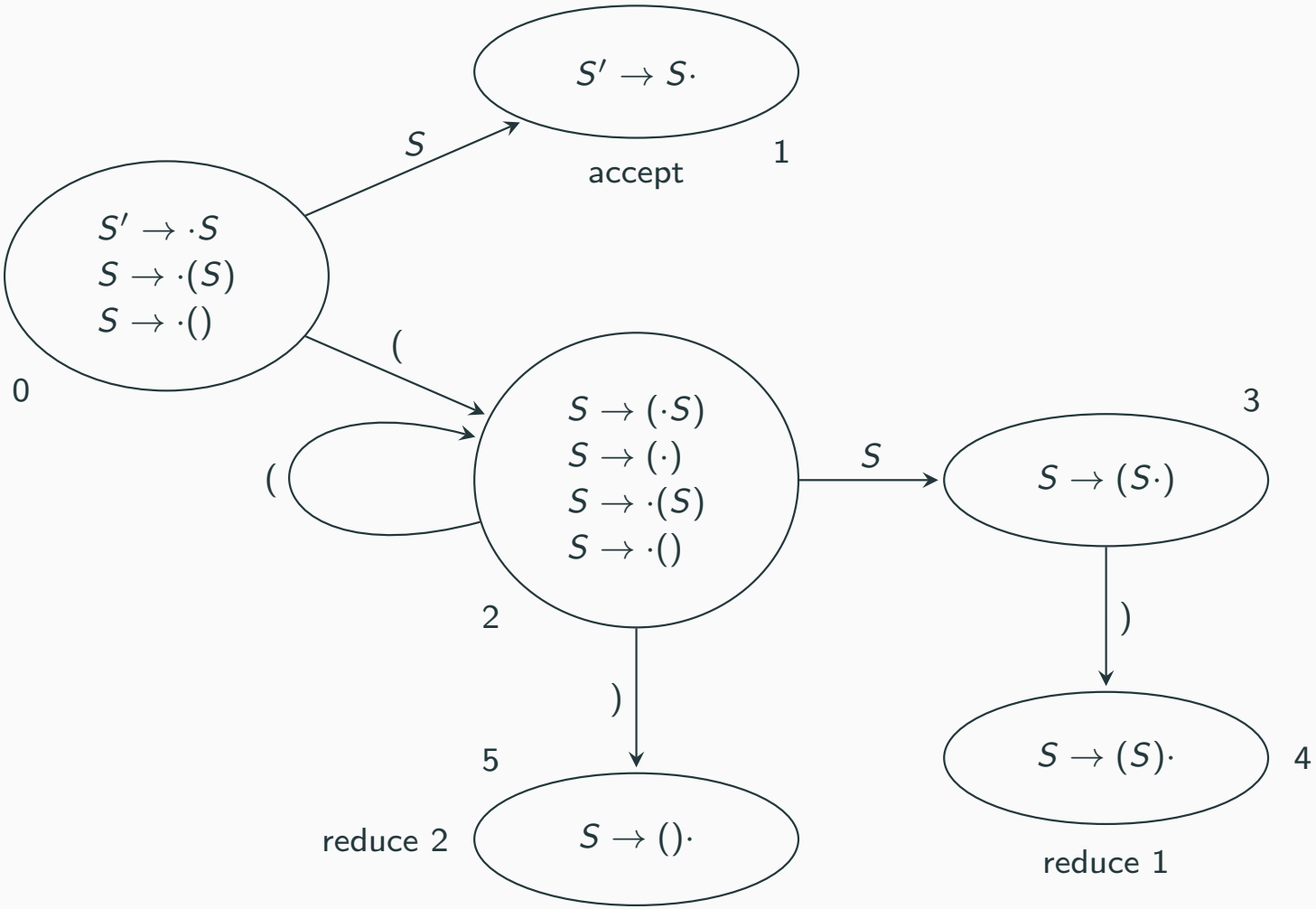
Inizializza $\mathcal{Q} = \{Clos(\{S' \to \cdot S\})\}$
Inizializza $\delta = \emptyset$
Ripeti fino a quando $\mathcal{Q}$ o δ vengono modificati:
  per ogni $I \in \mathcal{Q}$:
    per ogni item $A \to \alpha \cdot X\beta \in I$:
      $J = Goto(I, X)$
      aggiungi J ad $\mathcal{Q}$
      aggiungi $\delta(I, X) = J$ a δ
return $\mathcal{Q}$ e δ

Primo esempio

Presa la grammatica aumentata

$0.S' \to S \quad 1.S \to (S) \quad 2.S \to ()$,

il suo automa canonico LR(0) è



Primo esempio (2)

- Lo stato iniziale è
  $Clos(\{S' \to \cdot S\}) = \{S' \to \cdot S, S \to \cdot(S), S \to \cdot()\}$ = stato 0
- $Goto(0, ( ) = Clos(\{S \to (\cdot S), S \to (\cdot)\})$ = stato 2
- $Goto(2, ( ) = Clos(\{S \to (\cdot S), S \to (\cdot)\})$ = stato 2
- Osserva che ogni cammino "breve" dallo stato iniziale ad uno terminale corrisponde alla parte destra di una produzione:
  - Da 0 a 1 l'etichetta è S e la produzione è la 0.
  - Da 0 a 4 l'etichetta è (S) e la produzione è la 1.
  - Da 0 a 5 l'etichetta è () e la produzione è la 2.
- Osserva che ogni stato "indica" univocamente una azione da compiere:
  - per gli stati 0, 2 e 3, l'azione è shift,
  - per lo stato 1, l'azione è accept,
  - per gli stati 4 e 5, l'azione è reduce (1 e 2 rispettivamente).
  - **N.B.**: questo sarà vero per le grammatiche LR(0), ma non in generale.

Secondo esempio

Presa la grammatica aumentata

$$0.S' \to S \quad 1.S \to (S) \quad 2.S \to A \quad 3.A \to a,$$

il suo automa canonico LR(0) è

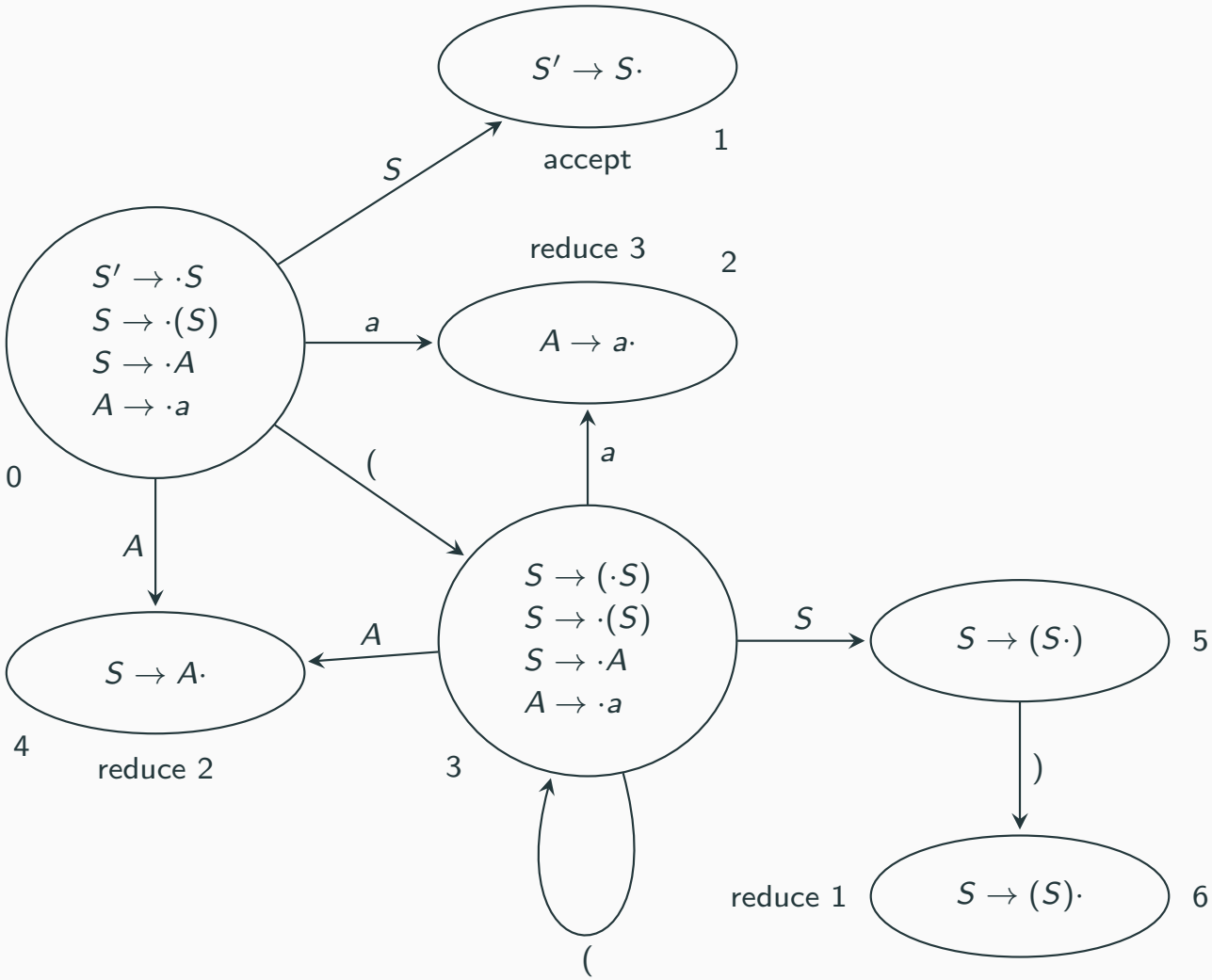


Tabella di parsing LR(0)

A partire dall'automa canonico LR(0), si costruisce la tabella di parsing LR(0), che il parser effettivamente userà per scegliere l'azione da compiere.
La tabella è una matrice bidimensionale M:

- **righe**: stati dell'automa canonico LR(0)
- **colonne**: $T \cup \{\$\}$ (parte "azione") e anche NT (parte "goto")
- **casella** $M[q, X]$ contiene le azioni che può compiere un parser LR(0) con q in cima alla pila degli stati e X è o il simbolo in input (per azione di shift o reduce) oppure un nonterminale (per goto)
  - Se $M[q, X]$ è "bianca"/vuota, allora errore
  - Se $M[q, X]$ contiene più azioni, allora conflitto (il parser non è deterministico)

Come si riempie la tabella di parsing LR(0)?

Per ogni stato q dell'automa canonico LR(0):

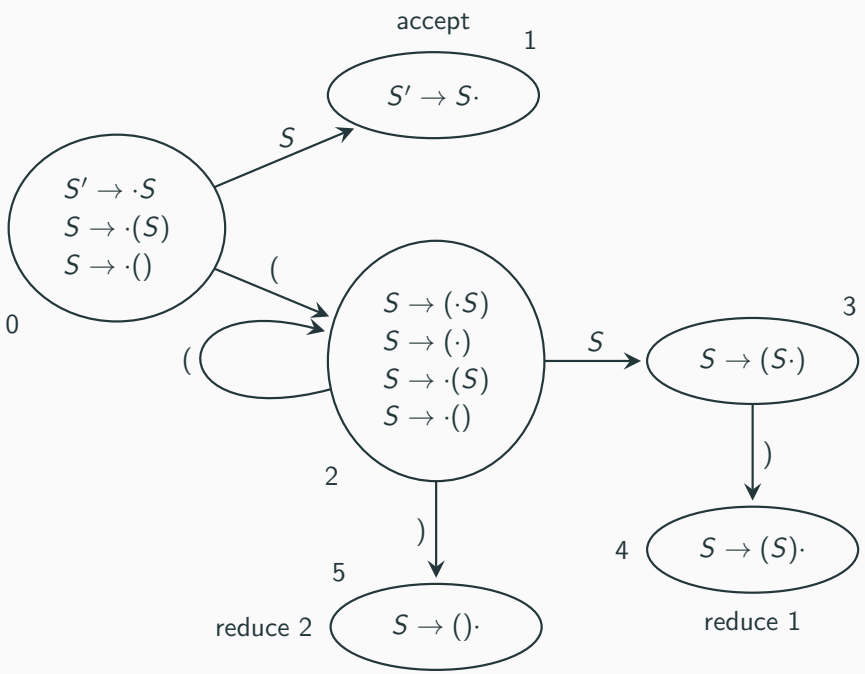
1. se $x \in T$ e $q \xrightarrow{x} t$ nell'automa LR(0), inserisci `shift t` in $M[q, x]$,
2. se $A \to \alpha\cdot \in q$ e $A \neq S'$, inserisci `reduce` $A \to \alpha$ in $M[q, x]$ per tutti gli $x \in T \cup \{\$\}$,
3. se $S' \to S\cdot \in q$, inserisci `accept` in $M[q, \$]$,
4. se $A \in NT$ e $q \xrightarrow{A} t$ nell'automa LR(0), inserisci `goto t` in $M[q, A]$.

Definizione: Una grammatica libera è di classe LR(0) se ogni casella nella tabella di parsing LR(0) contiene al più un elemento.

N.B.: Lo 0 in LR(0) sta ad indicare che le reduce vengono fatte senza considerare quale sia il prossimo simbolo in input; infatti, al punto 2, abbiamo scritto per tutti gli $x \in T \cup \{\$\}$.

Primo esempio

$0.S' \to S \quad 1.S \to (S) \quad 2.S \to ()$

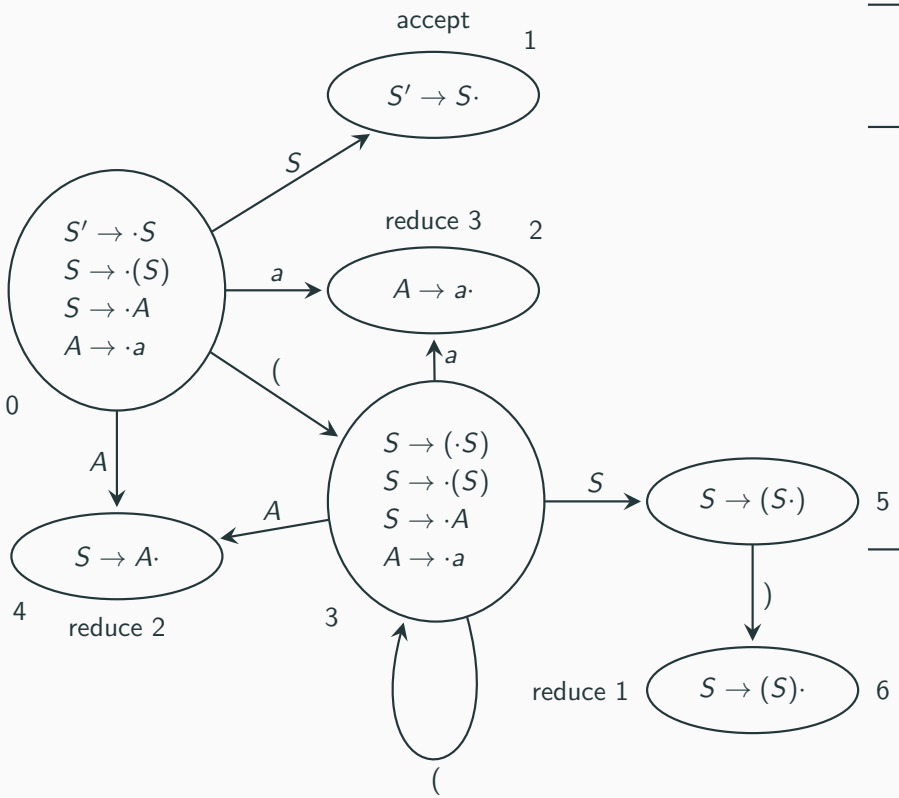


Stato	ACTION			GOTO
	(	)	$	S
0	s_2			g_1
1			acc	
2	s_2	s_5		g_3
3		s_4		
4	r_1	r_1	r_1	
5	r_2	r_2	r_2	

s_2 sta per shift 2, g_1 per goto 1, r_2 per reduce 2, *acc* per accept.
Non ci sono conflitti, quindi la grammatica è di classe LR(0).

Secondo esempio

$0.S' \to S \quad 1.S \to (S) \quad 2.S \to A \quad 3.A \to a$



Stato	ACTION				GOTO	
	a	(	)	$	S	A
0	s_2	s_3			g_1	g_4
1				acc		
2	r_3	r_3	r_3	r_3		
3	s_2	s_3			g_5	g_4
4	r_2	r_2	r_2	r_2		
5			s_6			
6	r_1	r_1	r_1	r_1		

Non ci sono conflitti, quindi la grammatica è di classe LR(0).

Come funziona il parser LR(0)?

- Ha a disposizione uno stack, dove impilare gli stati dell'automa canonico LR(0) che man mano va a toccare.
  - Inizialmente, la pila contiene lo stato iniziale q_0 dell'automa canonico LR(0), che rimane sempre in fondo alla pila.
- Esamina l'input $w\$$, un carattere alla volta.
  - Inizialmente, si posiziona sul primo carattere dell'input.
- Una configurazione del parser LR(0) è data dalla coppia $(q_0 q_1 \dots q_m, a_i a_{i+1} \dots a_k\$)$, dove $q_0 q_1 \dots q_m$ rappresenta lo stack degli stati (che cresce verso destra), mentre $a_i a_{i+1} \dots a_k\$$ rappresenta l'input non ancora letto.
  - Se indichiamo con X_i il simbolo che viene letto sull'automa canonico LR(0) passando dallo stato q_{i-1} allo stato q_i della pila (per $1 \leq i \leq m$), allora abbiamo che $X_1 X_2 \dots X_m a_i a_{i+1} \dots a_k$ rappresenta la forma sentenziale corrente della derivazione rightmost che il parser sta ricostruendo.

Come funziona il parser LR(0)? (2)

- Data la configurazione corrente $(q_0 q_1 \ldots q_m, a_i a_{i+1} \ldots a_k \$)$, il parser consulta la tabella di parsing LR(0) M, costruita a partire dall'automa canonico LR(0), come segue:
  - Se $M[q_m, a_i] =$ shift q, allora la nuova configurazione sarà $(q_0 q_1 \ldots q_m q, a_{i+1} \ldots a_k \$)$.
  - Se $M[q_m, a_i] =$ reduce $A \to \beta$, allora la nuova configurazione sarà $(q_0 q_1 \ldots q_{m-n} q, a_i a_{i+1} \ldots a_k \$)$, dove $n = |\beta|$ e $M[q_{m-n}, A] =$ goto q. Ovvero:
    - Prima fa per n volte pop dell'elemento top della pila (ora il nuovo stato top è q_{m-n});
    - poi consulta la tabella di parsing per vedere quale goto viene assegnato alla coppia q_{m-n}, A;
    - infine, fa push sullo stack dello stato così calcolato.
  - Se $M[q_m, a_i] =$ accept, allora ho terminato con successo (possibile solo se $a_i = \$$).
  - Se $M[q_m, a_i]$ è bianca, allora errore.

Primo esempio

$0.S' \to S \quad 1.S \to (S) \quad 2.S \to ()$

Stato	ACTION			GOTO
	(	)	$	S
0	s_2			g_1
1			acc	
2	s_2	s_5		g_3
3		s_4		
4	r_1	r_1	r_1	
5	r_2	r_2	r_2	

Step	Configurazione	Derivazione	Azione
1	$(0, (())\$),$	$\epsilon, (())\$$	shift 2
2	$(02, ())\$)$	$(, ())\$$	shift 2
3	$(022,))\$)$	$((,))\$$	shift 5
4	$(0225,)\$)$	$((),)\$$	reduce 2
5	$(023,)\$)$	$(S,)\$$	shift
6	$(0234, \$)$	$(S), \$$	reduce 1
7	$(01, \$)$	$S, \$$	accept

Secondo esempio

$0.S' \to S \quad 1.S \to (S) \quad 2.S \to A$
$3.A \to a$

Stato	ACTION				GOTO	
	a	(	)	$	S	A
0	s_2	s_3			g_1	g_4
1				acc		
2	r_3	r_3	r_3	r_3		
3	s_2	s_3			g_5	g_4
4	r_2	r_2	r_2	r_2		
5			s_6			
6	r_1	r_1	r_1	r_1		

Step	Configurazione	Derivazione	Azione
1	$(0, ((a))\$),$	$\epsilon, ((a))\$$	shift 3
2	$(03, (a))\$)$	$(, (a))\$$	shift 3
3	$(033, a))\$)$	$((, a))\$$	shift 2
4	$(0332,))\$)$	$((a,))\$$	reduce 3
5	$(0334,))\$)$	$((A,))\$$	reduce 2
6	$(0335,))\$)$	$((S,))\$$	shift 6
7	$(03356,)\$)$	$((S),)\$$	reduce 1
8	$(035,)\$)$	$(S,)\$$	shift 6
9	$(0356, \$)$	$(S), \$$	reduce 1
10	$(01, \$)$	$S, \$$	accept

Generico Parser LR – pseudocodice

```
Inizializza la pila con q0 #q0 stato iniziale automa
Inizializza ic con il primo carattere di input
while (true) do:
  q = top(pila)
  case M[q, ic] of:
    shift t:
      push t sulla pila; avanza ic sull'input
    accept:
      output 'accept'; return
    reduce A → α:
      output A → α
      pop |α| stati dalla pila
      q1 := top(pila)
      if M[q1, A] = goto q2
        then push q2 sulla pila, else errore
    else:
      errore #casella bianca
```

LR(0) and prefix-free languages

Definizione: Una grammatica libera G è di classe LR(0) se la sua tabella di parsing LR(0) non presenta conflitti (al più una azione in ogni casella). Un linguaggio libero L è di classe LR(0) se esiste G di classe LR(0) tale che $L = L(G)$.

Proposizione. Se L è libero deterministico e prefix-free, allora è di classe LR(0).

Esercizio: dimostra che $L = \{wcw^R \mid w \in (a|b)^*\}$ è di classe LR(0), costruendo un parser LR(0) per L.

Avvertenza: L'implicazione inversa non vale: esistono linguaggi di classe LR(0) che non sono prefix-free. Ad esempio, il linguaggio non prefix-free a^+ è generato dalla grammatica $S \to Sa \mid a$, la cui tabella di parsing LR(0) non presenta conflitti (verificatelo per esercizio!).

LR(0) and prefix-free languages (2)

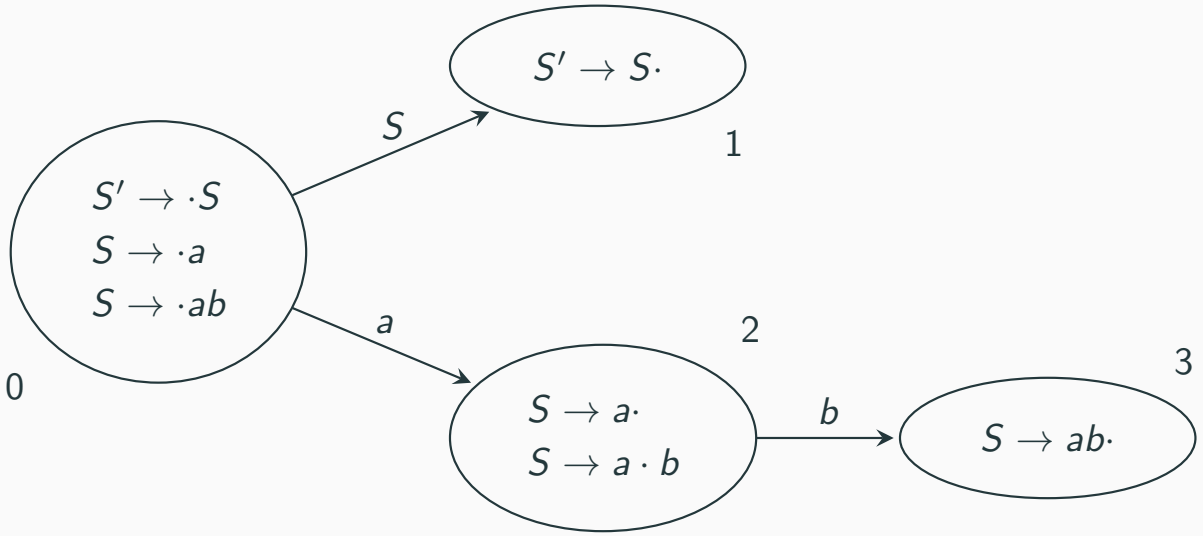
Avvertenza (2): Tuttavia, secondo alcune definizioni di grammatica LR(0), si esclude la possibilità che il simbolo iniziale S sia ricorsivo sinistro, per cui, sotto questa ipotesi, vale anche il rovescio della proposizione:
Se L è generato da una grammatica LR(0) (non-ricorsiva sul simbolo iniziale S), allora L è libero deterministico e prefix-free.
(Il contro-esempio mostrato in precedenza usa ricorsione sinistra sul simbolo iniziale).

In ogni caso, vale che se L è finito e non è prefix-free, allora L non è di classe LR(0). Ad esempio, $L = \{a, ab\}$. Quindi, tantissimi linguaggi liberi deterministici non sono di classe LR(0).

Parser SLR(1)

Primo esempio banale: $L = \{a, ab\}$

Consideriamo $0.S' \to S \quad 1.S \to a \quad 2.S \to ab$. Il suo automa canonico LR(0) è:



Stato	ACTION			GOTO
	a	b	$	S
0	s_2			g_1
1			acc	
2	r_1	r_1/s_3	r_1	
3	r_2	r_2	r_2	

Conflitto shift-reduce in $M[2, b]$. $L(G)$ non è prefix-free perché a è prefisso di ab. Osservate che se un linguaggio L contiene due stringhe x, y tale che x sia prefisso di y, l'automa canonico LR(0) presenta uno stato in cui ci deve essere una reduce per x e uno shift per la prosecuzione in y.

Ma possiamo risolvere il conflitto?

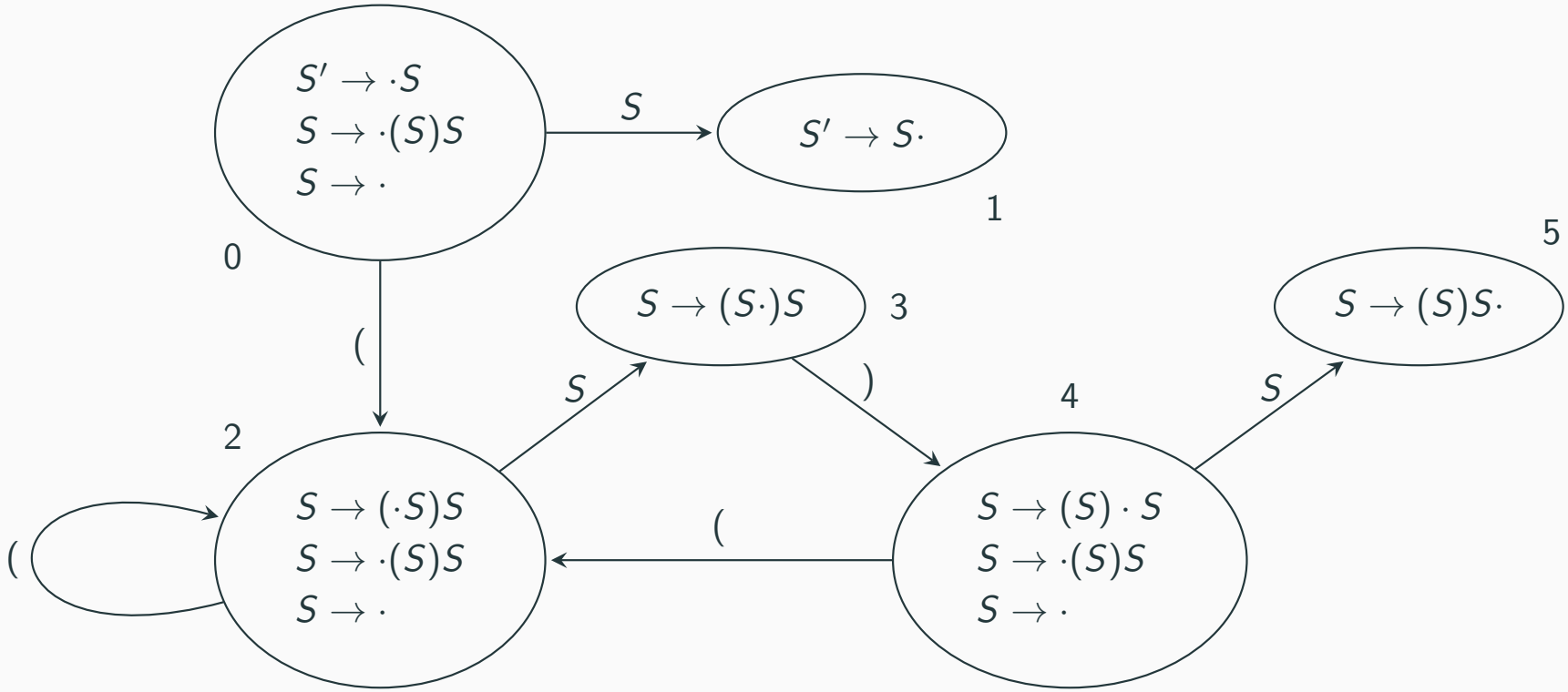
È possibile risolvere il conflitto? Usando una tecnica un po' più fine, che utilizza il look-ahead!
Nello stato 2, la reduce dovuta all'item di riduzione $S \rightarrow a\cdot$ ha senso farla solo quando il prossimo carattere in input appartiene al $Follow(S)$.
Se $b \in Follow(S)$, allora il conflitto è (può essere) reale, altrimenti no.
Nell'esempio, $Follow(S) = \{\$\}$, quindi risolviamo il conflitto a favore dello shift. In altre parole, possiamo riempire la tabella di parsing con un algortimo più fine di quello visto per il caso LR(0), second il quale le reduce vengono messe in tabella solo in corrispondenza dei simboli che appartengono al follow della testa.

Stato	ACTION			GOTO
	a	b	$	S
0	s_2			g_1
1			acc	
2		s_3	r_1	
3			r_2	

Tabella di parsing SLR(1), dove "S" sta per "simple" e 1 sta ad indicare che (implicitamente) usiamo un simbolo di look-ahead per decidere quando fare le reduce.

Esempio: parentesi bilanciate

Consideriamo $0.S' \to S \quad 1.S \to (S)S \quad 2.S \to \epsilon$, che è la grammatica che genera le parentesi bilanciate. Il suo automa canonico LR(0) è:



Osservate che negli stati 0, 2 e 4 c'è l'item di riduzione $S \to \cdot$ e anche l'item di shift $S \to \cdot(S)S$. Quindi, sicuramente, la tabella di parsing LR(0) presenta conflitti.

Esempio: parentesi bilanciate

Tabella di parsing LR(0)

Stato	ACTION			GOTO
	(	)	$	S
0	s_2/r_2	r_2	r_2	g_1
1			acc	
2	s_2/r_2	r_2	r_2	g_3
3		s_4		
4	s_2/r_2	r_2	r_2	g_5
5	r_1	r_1	r_1	

Tabella di parsing SLR(1)

Stato	ACTION			GOTO
	(	)	$	S
0	s_2	r_2	r_2	g_1
1			acc	
2	s_2	r_2	r_2	g_3
3		s_4		
4	s_2	r_2	r_2	g_5
5		r_1	r_1	

$Follow(S) = \{\$,)\}$

Quindi G è di classe SLR(1), ma non di classe LR(0).

Osservazione

- Se G ha ϵ-produzioni, allora G difficilmente è di classe LR(0):
  - in uno stato con item $A \to \cdot$ non posso avere item del tipo $B \to \alpha \cdot a\beta$, altrimenti avrei dei conflitti shift/reduce;
  - ovvero se $A \to \cdot$ è presente in uno stato, allora tutti gli altri item in quello stato, se ce ne sono, devono avere il punto $\cdot$ davanti a nonterminali!
- Esercizio: verifica che la grammatica $S \to Sa \mid \epsilon$ è di classe LR(0).

Come funziona il parser SLR(1)?

$0.S' \to S \quad 1.S \to (S)S \quad 2.S \to \epsilon$

Stato	ACTION			GOTO
	(	)	$	S
0	s_2	r_2	r_2	g_1
1			acc	
2	s_2	r_2	r_2	g_3
3		s_4		
4	s_2	r_2	r_2	g_5
5		r_1	r_1	

Step	Configurazione	Derivazione	Azione
1	$(0, ()()\$),$	$\epsilon, ()()\$$	shift 2
2	$(02,)()\$)$	$(,)()\$$	reduce 2
3	$(023,)()\$)$	$(S,)()\$$	shift 4
4	$(0234, ()\$)$	$(S), ()\$$	shift 2
5	$(02342,)\$)$	$(S)(,)\$$	reduce 2
6	$(023423, \$)$	$(S)(S,)\$$	shift 4
7	$(0234234, \$)$	$(S)(S), \$$	reduce 2
8	$(02342345, \$)$	$(S)(S)S, \$$	reduce 1
9	$(02345, \$)$	$(S)S, \$$	reduce 1
10	$(01, \$)$	$S, \$$	accept

$S \Rightarrow (S)S \Rightarrow (S)(S)S \Rightarrow (S)(S) \Rightarrow (S)() \Rightarrow ()()$

Esercizio corposo

(0) $E' \to E$ (1) $E \to E + T$ (2) $E \to T$

(3) $T \to T * A$ (4) $T \to A$ (5) $A \to a$ (6) $A \to (E)$

- Costruire l'automa canonico LR(0);
- costruire la tabella di parsing LR(0);
- verificare che ci sono conflitti;
- modificare la tabella di parsing in SLR(1).

L'automa ha 12 stati e quello iniziale contiene i seguenti 7 item:

$E' \to \cdot E \quad E \to \cdot E + T \quad E \to \cdot T \quad T \to \cdot T * A \quad T \to \cdot A \quad A \to \cdot a \quad A \to \cdot (E)$

Tabella di parsing SLR(1)

SLR(1) sta per:
Simple, **L**eft-to-right, **R**ightmost, **1** simbolo di look-ahead.
La tabella ha per

- righe: stati dell'automa canonico LR(0); e per
- colonne: $T \cup \{\$\} \cup NT$.

Come si riempie la tabella? Per ogni stato q dell'automa canonico LR(0):

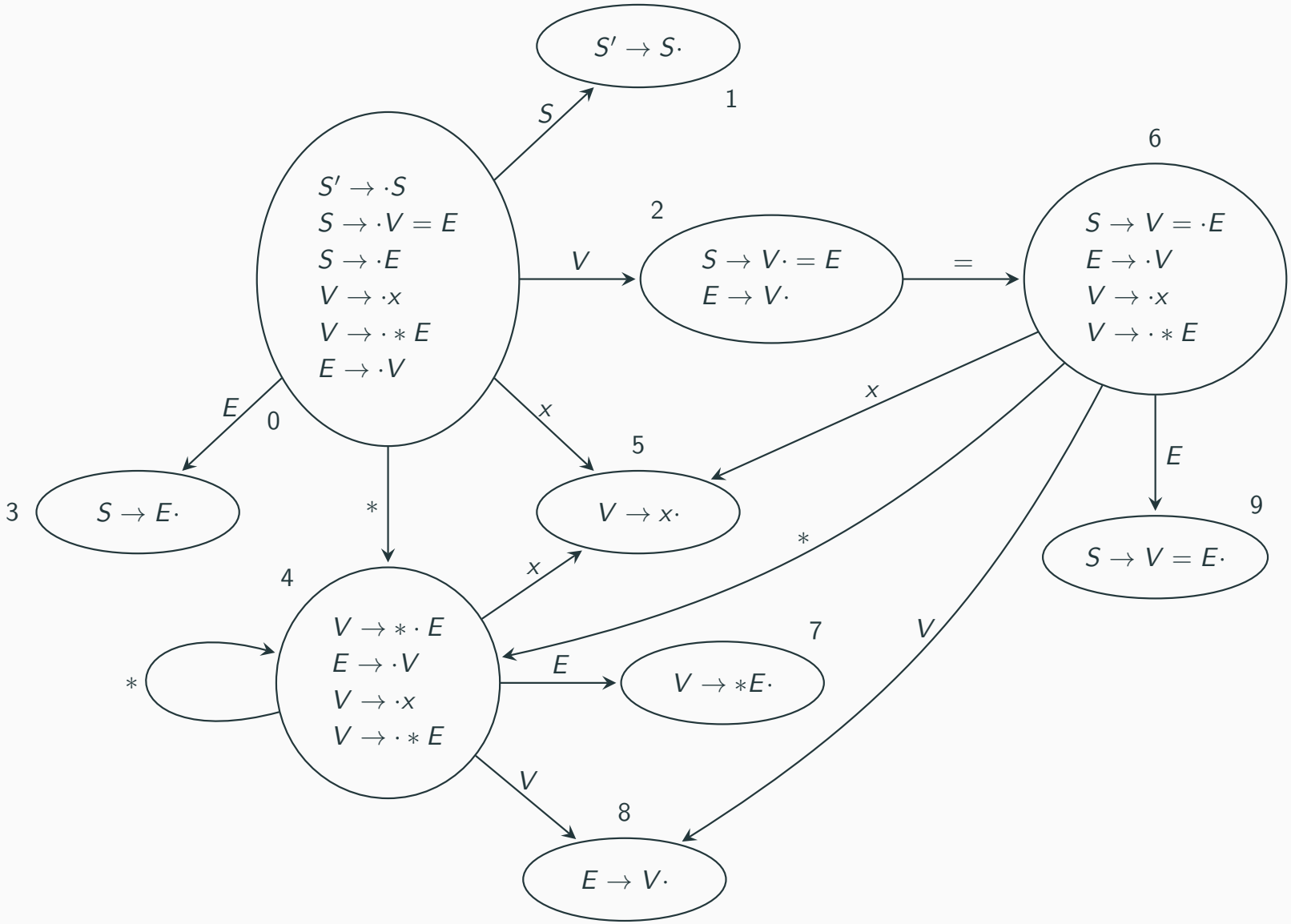
- se $x \in T$ e $q \xrightarrow{x} t$ nell'automa LR(0), inserisci `shift t` in $M[q, x]$
- se $A \to \alpha\cdot \in q$ e $A \neq S'$, inserisci `reduce` $A \to \alpha$ in $M[q, x]$ per tutti gli $x \in \mathit{Follow}(A)$
- se $S' \to S\cdot \in q$, inserisci `accept` in $M[q, \$]$
- se $A \in NT$ e $q \xrightarrow{A} t$ nell'automa LR(0), inserisci `goto t` in $M[q, A]$

Per LR(0), anziché "per tutti gli $x \in \underline{\mathit{Follow}(A)}$" c'era per tutti gli $x \in \underline{T \cup \{\$\}}$: l'effetto è limitare l'uso della reduce solo a casi plausibili.

Parser LR(1)

Grammatica non SLR(1)!

$0.S' \to S \quad 1.S \to V = E \quad 2.S \to E \quad 3.E \to V \quad 4.V \to x \quad 5.V \to *E$



Perché non è SLR(1)?

Esercizio: costruire la tabella di parsing SLR(1).

- Focalizziamoci sullo stato 2: $\{S \to V\cdot = E, E \to V\cdot\}$.
- Presente conflitto shift/reduce, anche in SLR(1): $M[2, =] = \{s_6, r_3\}$. La reduce r_3 è presente perché $= \in \mathit{Follow}(E)$ (per il fatto che $V \to *E$, si ha che $\mathit{Follow}(V) \subseteq \mathit{Follow}(E)$).
- Ma se sappiamo con certezza che il carattere successivo è davvero "=", allora la reduce $E \to V\cdot$ non va considerata, perché non è possibile derivare $S \Rightarrow^* E = \ldots$ Allora ha senso solo fare lo shift!

Osservazione: $= \in \mathit{Follow}(E)$ vuol dire che esiste una derivazione tale che $S \Rightarrow^* \alpha E = \beta$, ma a noi serve sapere se $S \Rightarrow^* E = \beta$ perché questa è la derivazione "corrente".
N.B.: $S \Rightarrow V = E \Rightarrow *E = E$, per cui effettivamente $= \in \mathit{Follow}(E)$, ma è impossibile derivare $S \Rightarrow^* E = \ldots$, per cui nello stato 2, se in input c'è "=", devo sicuramente fare lo shift a 6.

E allora?

Da questo esempio, si capisce che bisogna ridurre i casi in cui si possa applicare una reduce a casi ancora più plausibili di quanto dica $Follow(E)$!
Vorremmo legare le reduce solo a quei follow che effettivamente possono comparire nella derivazione corrente. Per far questo, introduciamo l'item LR(1).
Un **item LR(1)** è una coppia formata da:

- Un item LR(0), detto nucleo/core, e da
- un simbolo di look-ahead in $T \cup \{\$\}$.

Con gli item LR(1), costruiremo un automa canonico LR(1), più dettagliato dell'automa canonico LR(0).
Per la grammatica vista in precedenza, l'automa canonico LR(1) presenta uno stato 2 siffatto: $\{[S \to V \cdot = E, \$], [E \to V \cdot, \$]\}$. Qui non c'è nessun conflitto, perché la reduce verrà fatta solo per il simbolo di look-ahead, che è \$, quindi, se leggo $=$, faccio lo shift.

Intuizione

Se l'automa canonico LR(1), che definiremo tra poco, è in uno stato che contiene l'item LR(1) $[A \to \alpha \cdot \beta, x]$ (cioè tale stato è in cima alla pila degli stati), allora:

- il parser LR(1) sta cercando di riconoscere la maniglia $\alpha\beta$;
- di essa, α è già sulla pila dei simboli;
- sull'input si aspetta una stringa derivabile da βx
  - cioè x può davvero essere fatta in questa derivazione:
    - quando uso il $\mathit{Follow}(A)$ in SLR(1), so solo che esiste una derivazione che produce $x \in \mathit{Follow}(A)$, ma non che x sia proprio il follow nella derivazione che sto esaminando.

Se lo stato dell'automa canonico LR(1) in cima alla pila contiene l'item LR(1) $[A \to \alpha\cdot, x]$, allora il parser può fare la reduce $A \to \alpha$ solo se il prossimo input è proprio x.

Automa Canonico LR(1)

L'automa canonico LR(1) viene costruito usando le funzioni ausiliarie $Clos(I)$ e $Goto(I, X)$ dove I è un insieme di item LR(1) e $X \in T \cup NT$.

```
def Clos(I):
  ripeti fino a quando I viene modificato:
    per ogni item [A → α · Xβ, a] ∈ I:
      per ogni produzione X → γ:
       per ogni b ∈ First(βa)
        aggiungi [X → · γ, b] ad I
  return I

def Goto(I, X):
  inizializza J = ∅
  per ogni item [A → α · Xβ, a] ∈ I:
    aggiungi [A → αX · β, a] a J
  return Clos(J)
```

$Goto(I, X)$ non considera il look-ahead, cioè agisce solo nella parte core/LR(0) dell'item LR(1).

Costruzione dell'Automa Canonico LR(1)

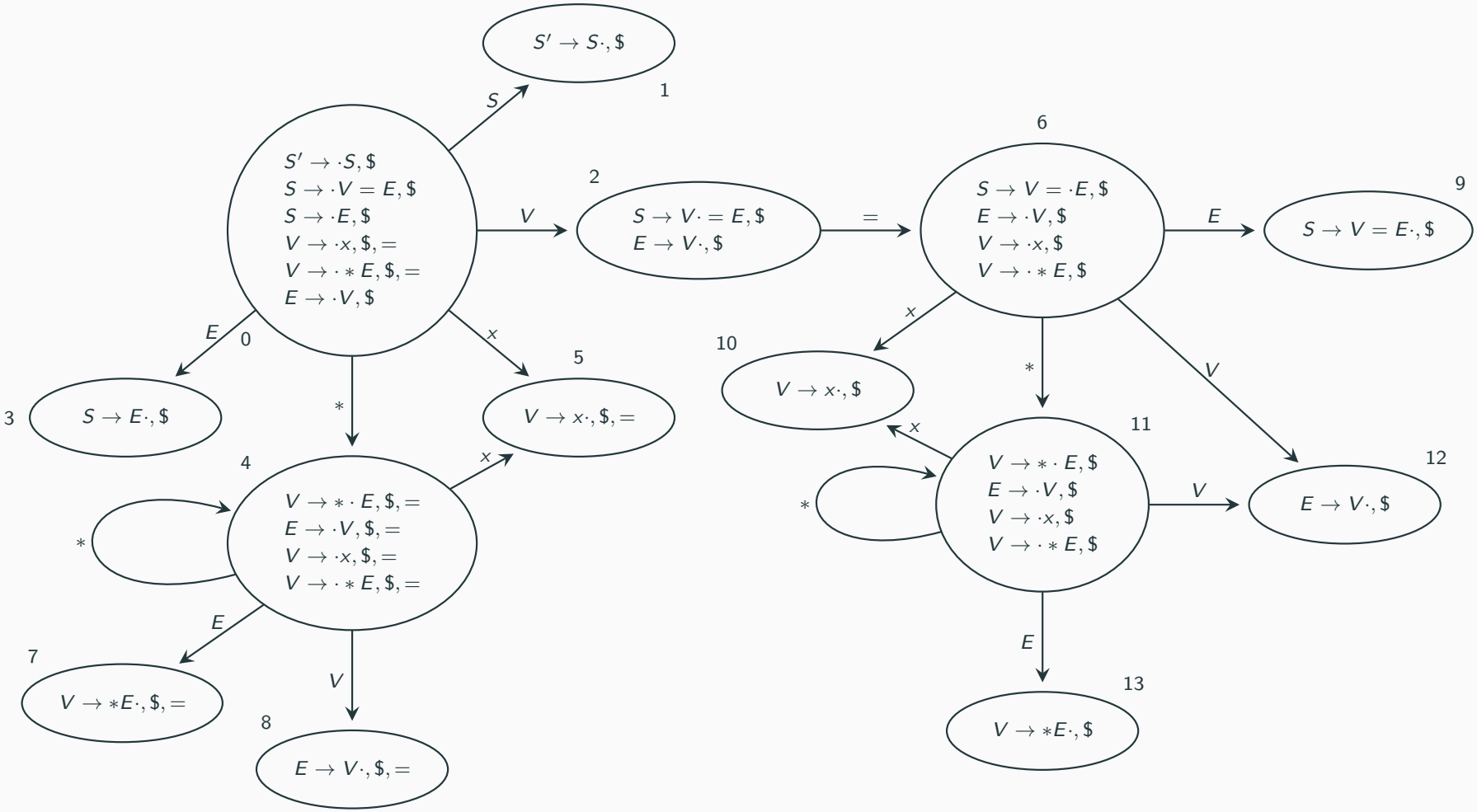
Inizializza $\mathcal{Q} = \{Clos(\{[S' \to \cdot S, \$]\})\}$
Inizializza $\delta = \emptyset$
Ripeti fino a quando $\mathcal{Q}$ o δ vengono modificati:
  per ogni $I \in \mathcal{Q}$:
    per ogni item $[A \to \alpha \cdot X\beta, a] \in I$:
      $J = Goto(I, X)$
      aggiungi J ad $\mathcal{Q}$
      aggiungi $\delta(I, X) = J$ a δ
return $\mathcal{Q}$ e δ

Si inizia dallo stato $Clos(\{[S' \to \cdot S, \$]\})$. Nota che il look-ahead è \$ perché questo è il simbolo di fine input.

Notazione: Se in uno stato abbiamo più item con lo stesso core $[A \to \alpha \cdot X\beta, a_1], \ldots [A \to \alpha \cdot X\beta, a_k]$, allora li accorpiamo in un unico item con più look-ahead $[A \to \alpha \cdot X\beta, a_1, \ldots, a_k]$. Per semplicità, omettiamo spesso le parentesi quadre.

Esempio: grammatica non SLR(1)

$0.S' \to S \quad 1.S \to V = E \quad 2.S \to E \quad 3.E \to V \quad 4.V \to x \quad 5.V \to *E$



Coppie di stati con medesimo core/LR(0): 4-11, 5-10, 7-13, 8-12.

Tabella di parsing LR(1)

Per ogni stato q dell'automa canonico LR(1):

- se $x \in T$ e $q \xrightarrow{x} t$ nell'automa LR(1), inserisci `shift t` in $M[q, x]$
- se $[A \to \alpha\cdot, x] \in q$ e $A \neq S'$, inserisci `reduce`$A \to \alpha$ in $M[q, x]$ (solo per x del look-ahead!)
- se $[S' \to S\cdot \in q, \$]$, inserisci `accept` in $M[q, \$]$
- se $A \in NT$ e $q \xrightarrow{A} t$ nell'automa LR(1), inserisci `goto t` in $M[q, A]$

Ogni casella rimasta vuota è un errore.

Definizione: Una grammatica libera G è di classe LR(1) se ogni casella della sua tabella di parsing LR(1) ha al più un elemento (no conflitti).

Esempio: tabella di parsing LR(1)

$0.S' \to S \quad 1.S \to V = E \quad 2.S \to E \quad 3.E \to V \quad 4.V \to x \quad 5.V \to *E$

Stato	ACTION				GOTO		
	x	*	=	$	S	E	V
0	s_5	s_4			g_1	g_3	g_2
1				acc			
2			s_6	r_3			
3				r_2			
4	s_5	s_4				g_7	g_8
5			r_4	r_4			
6	s_{10}	s_{11}				g_9	g_{12}
7			r_5	r_5			
8			r_3	r_3			
9				r_1			
10				r_4			
11	s_{10}	s_{11}				g_{13}	g_{12}
12				r_3			
13				r_5			

Non ci sono conflitti, quindi la grammatica è di classe LR(1).

Come funziona il parser LR(1)?

$0.S' \to S \quad 1.S \to V = E \quad 2.S \to E \quad 3.E \to V \quad 4.V \to x \quad 5.V \to *E$

Step	Configurazione	Derivazione	Azione
1	$(0, *x = x\$)$,	$\epsilon, *x = x\$$	shift 4
2	$(04, x = x\$)$	$*, x = x\$$	shift 5
3	$(045, = x\$)$	$*x, = x\$$	reduce 4
4	$(048, = x\$)$	$*V, = x\$$	reduce 3
5	$(047, = x\$)$	$*E, = x\$$	reduce 5
6	$(02, = x\$)$	$V, = x\$$	shift 6
7	$(026, x\$)$	$V =, x\$$	shift 10
8	$(026\text{-}10, \$)$	$V = x, \$$	reduce 4
9	$(026\text{-}12, \$)$	$V = V, \$$	reduce 3
10	$(0269, \$)$	$V = E, \$$	reduce 1
11	$(01, \$)$	$S, \$$	accept

$S \Rightarrow V = E \Rightarrow V = V \Rightarrow V = x \Rightarrow *E = x \Rightarrow *V = x \Rightarrow *x = x$

Parser LALR(1)

Parser LALR(1)

"LALR(1)" sta per Look-Ahead LR(1)

- Approccio LR(1): tabelle di parsing molto grandi (centinaia di migliaia di stati per linguaggi di media grandezza), quindi poco pratico.
- Approccio LALR(1): buon compromesso tra semplicità (e compattezza) di SLR(1) e selettività di LR(1).

Come si ottiene il parser LALR(1)? Osserva che

- Nucleo di uno stato dell'automa canonico LR(1) = insieme di item LR(0) ottenuto dimenticando i look-ahead dagli item LR(1).
  - Nucleo di stato LR(1) = stato dell'automa canonico LR(0).
- Le transizioni dell'automa LR(1) dipendono solo dal nucleo; infatti, la funzione $Goto(I, X)$ usa di I solo la parte nucleo/LR(0) dell'item.

La tabella di parsing LALR(1) si ottiene da quella LR(1) fondendo insieme gli stati con lo stesso nucleo:

- tante righe quanti gli stati dell'automa canonico LR(0), quindi tante righe come per la tabella SLR(1),
- ma molte meno reduce della tabella SLR(1).

Riprendiamo l'esempio

$0.S' \to S \quad 1.S \to V = E$

$2.S \to E \quad 3.E \to V \quad 4.V \to x$

$5.V \to *E$

Tabella LR(1)

Stato	ACTION				GOTO		
	x	*	=	$	S	E	V
0	s_5	s_4			g_1	g_3	g_2
1				acc			
2			s_6	r_3			
3				r_2			
4	s_5	s_4				g_7	g_8
5			r_4	r_4			
6	s_{10}	s_{11}				g_9	g_{12}
7			r_5	r_5			
8			r_3	r_3			
9				r_1			
10				r_4			
11	s_{10}	s_{11}				g_{13}	g_{12}
12				r_3			
13				r_5			

Stati con stesso nucleo: 4-11, 5-10, 7-13, 8-12.

Tabella LALR(1)

Stato	ACTION				GOTO		
	x	*	=	$	S	E	V
0	s_5	s_4			g_1	g_3	g_2
1				acc			
2			s_6	r_3			
3				r_2			
4	s_5	s_4				g_7	g_8
5			r_4	r_4			
6	s_5	s_4				g_9	g_8
7			r_5	r_5			
8			r_3	r_3			
9				r_1			

Non ci sono conflitti, quindi G è LALR(1).

Tabella grande come quella di SLR(1) per G, ma questa non ha conflitti!

LR(1) e LALR(1)

Se G è LR(1) e anche LALR(1), allora il parser LR(1) e quello LALR(1) si mimano perfettamente su input corretti.
Su input erronei, LALR(1) può fare qualche riduzione in più prima di accorgersi dell'errore. Ad esempio, per la G di sopra:

parser LR(1)

$(0, x = x = \$)$
$(05, = x = \$)$
$(02, = x = \$)$
$(026, x = \$)$
$(026\text{-}10, = \$)$

errore

parser LALR(1)

$(0, x = x = \$)$
$(05, = x = \$)$
$(02, = x = \$)$
$(026, x = \$)$
$(0265, = \$)$
$(0268, = \$)$
$(0269, = \$)$

errore

L'automa LALR(1) non farà mai degli shift in più dell'automa LR(1) per un certo input. Quindi:

$\Rightarrow$ i due parser consumano la stessa porzione di input prima di rilevare l'errore;

$\Rightarrow$ LALR(1) è corretto come sostituto di LR(1).

Passando da LR(1) a LALR(1)

Supponiamo di avere una grammatica G di classe LR(1) e vediamo se è anche di classe LALR(1).

- La fusione di due stati LR(1) con lo stesso core può causare conflitti.
- Sono possibili solo nuovi conflitti reduce/reduce.
- Infatti, supponiamo che in q, stato ottenuto per fusione di due stati LR(1) q_1, q_2, presenti un conflitto shift/reduce. Allora, esistono in q un item $[A \to \alpha\cdot, a]$ ed un item $[B \to \beta \cdot a\gamma, b]$. Supponiamo, senza perdita di generalità, che $[A \to \alpha\cdot, a] \in q_1$. Allora $[A \to \alpha\cdot, a]$ e $[B \to \beta \cdot a\gamma, c]$ (per qualche c) appartengono a q_1:
  - $\Rightarrow$ pure q_1 in LR(1) avrebbe un conflitto shift-reduce, contro l'ipotesi che la tabella LR(1) non presenti conflitti!
  - $\Rightarrow$ se LR(1) è senza conflitti, LALR(1) potrebbe solo presentare conflitti reduce/reduce!

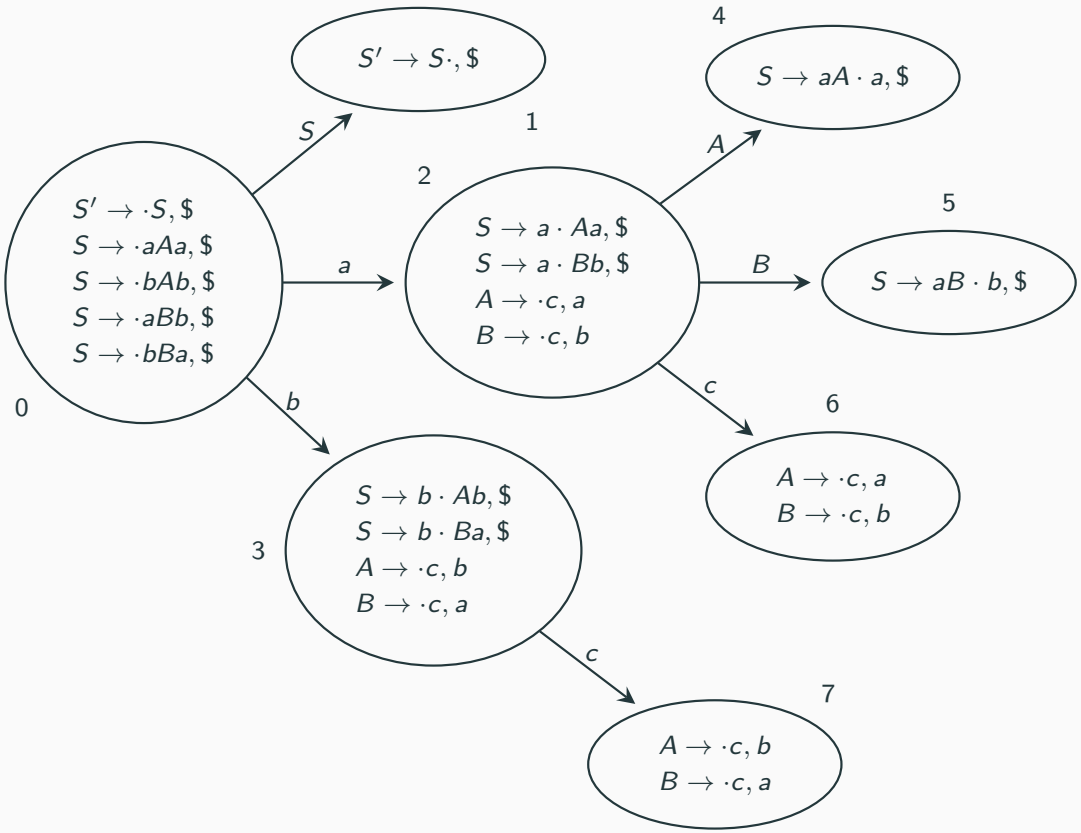
Se si generano conflitti, allora G non è LALR(1), pur essendo LR(1).

Esempio: G è LR(1) ma non LALR(1)

$$S \to aAa \mid bAb \mid aBb \mid bBa$$

$$A \to c \quad B \to c$$

Porzione **incompleta** dell'automa canonico LR(1):mancano alcuni stati e transizioni



Se completate l'automa LR(1), vi renderete conto che non ci sono conflitti.

Esempio: G è LR(1) ma non LALR(1)

- Focalizziamoci sugli stati 6 e 7.
  - Hanno lo stesso core/LR(0).
  - Ma se li fondo assieme, ottengo uno stato con questi quattro item: $[A \to c\cdot, a]$, $[A \to c\cdot, b]$, $[B \to c\cdot, a]$ e $[B \to c\cdot, b]$.
- Si viene a creare un conflitto reduce-reduce:
  - sulla tabella LALR(1), per lo stato ottenuto dalla fusione di 6 e 7, in corrispondenza di a e b, avrò sia la reduce $A \to c$, sia la reduce $B \to c$.

Quindi G è LR(1), ma non LALR(1)!

LALR(1) e LR(1)

Abbiamo descritto come costruire una tabella di parsing LALR(1) a partire da una tabella di parsing LR(1).

Tuttavia è possibile costruire la tabella di parsing LALR(1) anche senza dover prima generare l'automa canonico LR(1), ma direttamente dall'automa canonico LR(0) con maggiore efficienza nella costruzione.

Questa tecnica, che non vedremo, è quella usata dal generatore di parser YACC, che presenteremo brevemente in una prossima lezione.

Ricapitolando

Come si riempiono le tabelle di parsing LR?

- $M[q, x] =$ shift t, se $q \xrightarrow{x} t$ nell'automa canonico LR(0), con $x \in T$:
  - sia per LR(0), sia per SLR(1), sia per LR(1) – in quest'utlimo caso, $q \xrightarrow{x} t$ è una transizione dell'automa canonico LR(1).
- $M[q, \$] =$ *accept*, se $S' \to S\cdot$ appartiene a q nell'automa canonico LR(0):
  - sia per LR(0), sia per SLR(1), sia per LR(1) – in quest'utlimo caso, q è uno stato dell'automa canonico LR(1).
- $M[q, A] =$ goto t, se $q \xrightarrow{A} t$ nell'automa canonico LR(0), con $A \in NT$:
  - sia per LR(0), sia per SLR(1), sia per LR(1) – in quest'utlimo caso, $q \xrightarrow{A} t$ è una transizione dell'automa canonico LR(1).
- $M[q, x] =$ reduce $A \to \alpha$, se $A \to \alpha\cdot$ appartiene a q nell'automa canonico LR(0):
  - LR(0): $\forall x \in T \cup \{\$\}$, SLR(1): $\forall x \in \mathit{Follow}(A)$, LR(1): se $[A \to \alpha\cdot, x]$ appartiene a q nell'automa canonico LR(1), cioè solo per il look-ahead x.
- Per LALR(1), si prende la tabella LR(1) e si fondono gli stati con lo stesso "core LR(0)".